The Segment Tree – *motivation*

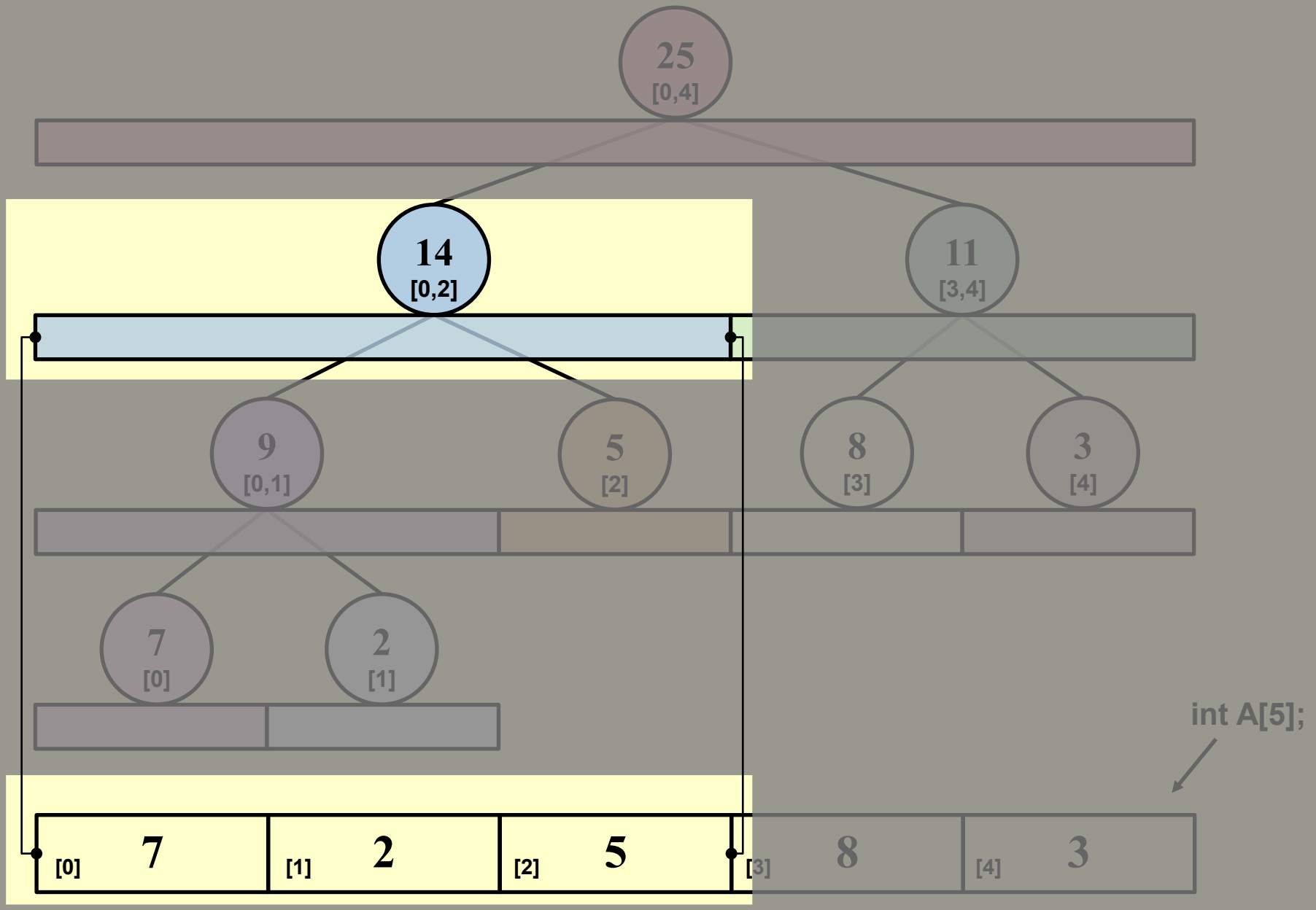
Given an array A[1000000], and we need to *frequently* calculate the sum of the numbers in an arbitrary *range* [L, R].


```
ElementType  Query ( ElementType A[ ],  int L, int R )
{
    ElementType sum = 0;
    for  ( int i = L; i <= R; ++i )
        sum += A[i];
    return sum;
}
```

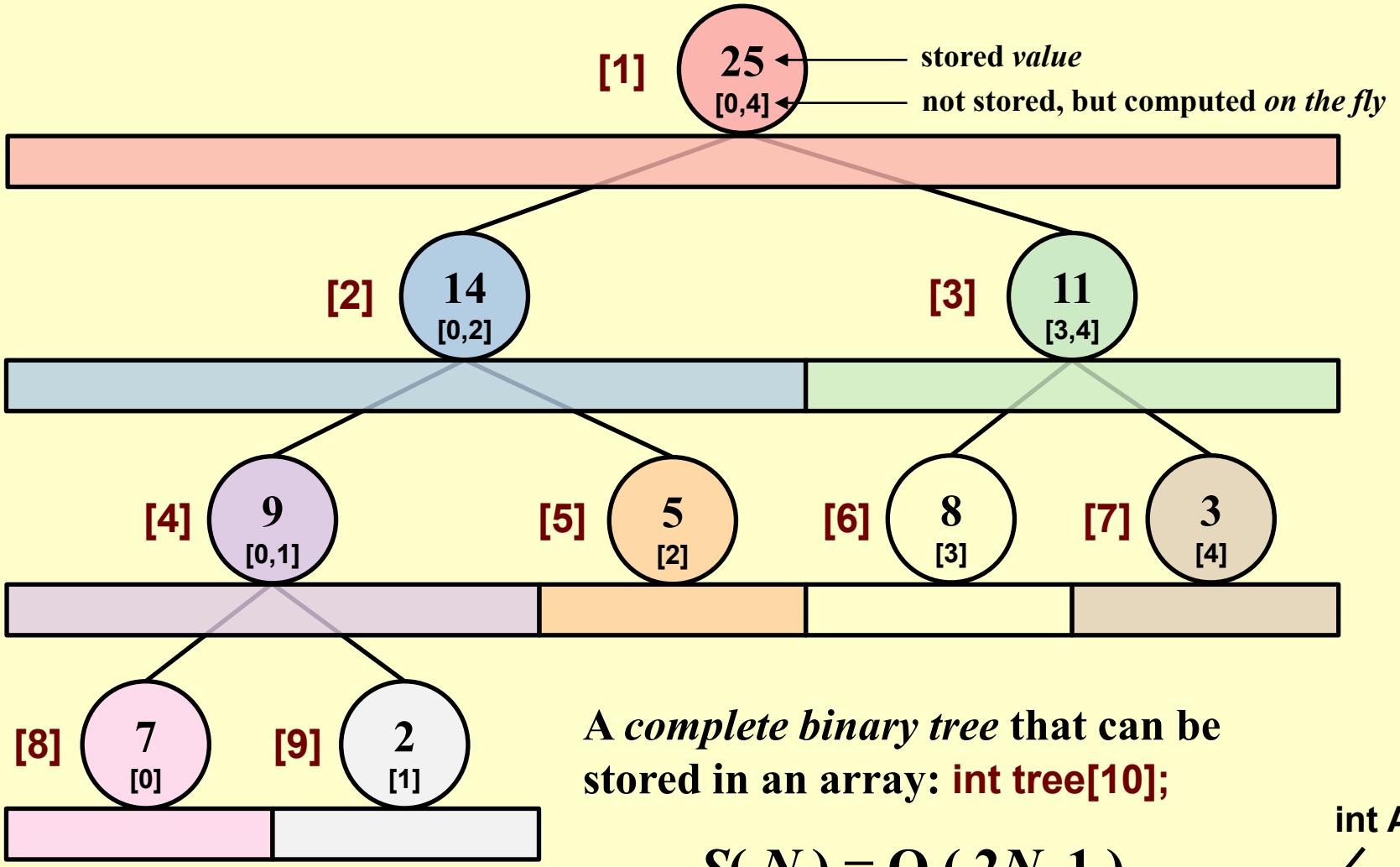
$T(N) = O(N)$

👎 as a frequent operation

The Segment Tree – *structure*



The Segment Tree – *how to build?*



A *complete binary tree* that can be stored in an array: int tree[10];

$$S(N) = O(2N-1)$$

int A[5];

[0] 7	[1] 2	[2] 5	[3] 8	[4] 3

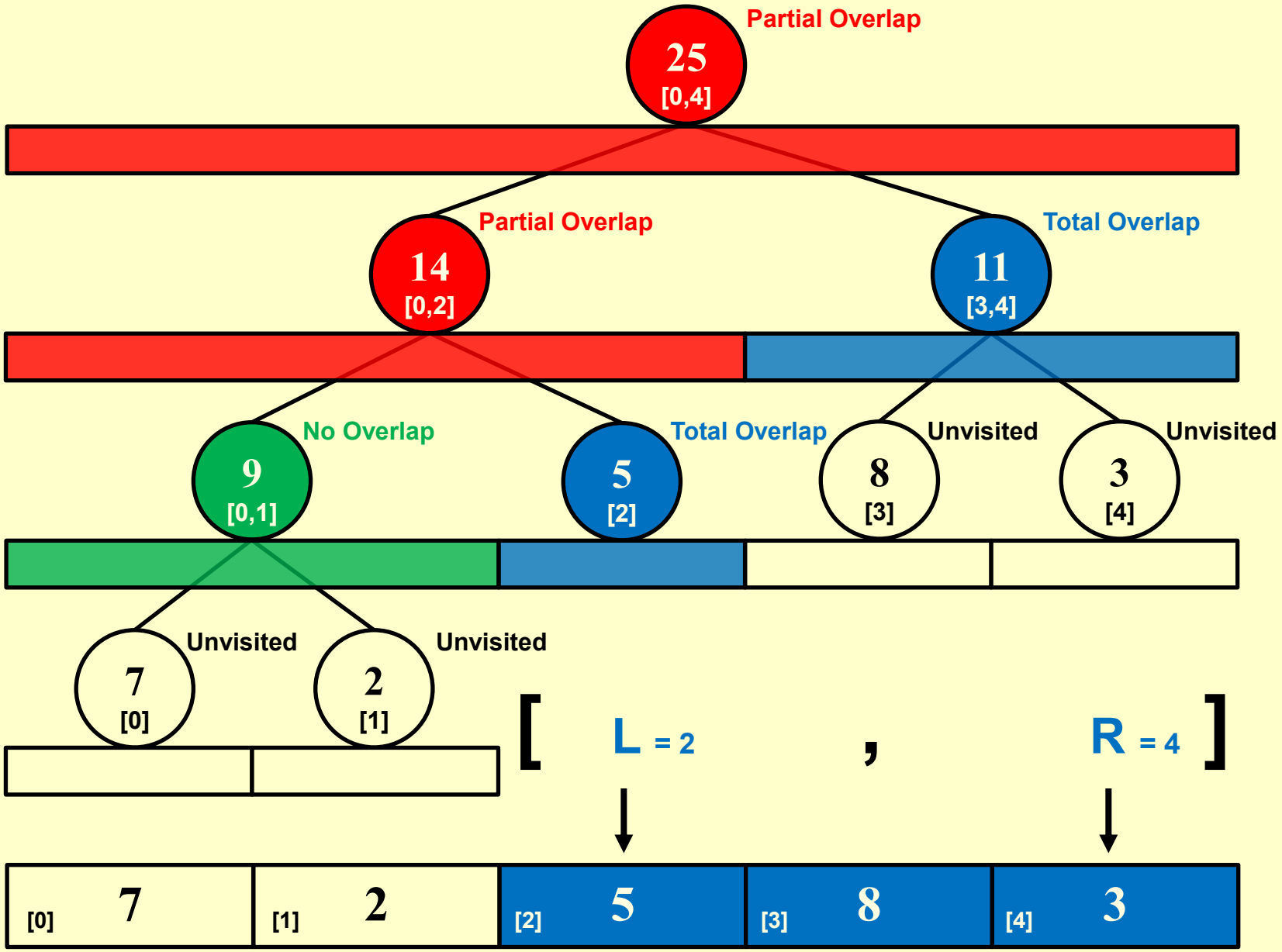
The Segment Tree – *how to build?*

```
void Build ( int node, int start, int end )
{
    // Base Case: Leaf Node
    if ( start == end ) {
        tree[ node ] = A[ start ];
        return;
    }

    // Recursive Step: Split and Build Children
    int mid = ( start + end ) / 2;
    Build ( 2 * node, start, mid );
    Build ( 2 * node + 1, mid + 1, end );

    // Merge Logic: Sum of children
    tree[ node ] = tree[ 2 * node ] + tree[ 2 * node + 1 ];
}
```

$T(N) = O(N)$

but run only once

The Segment Tree – *query*

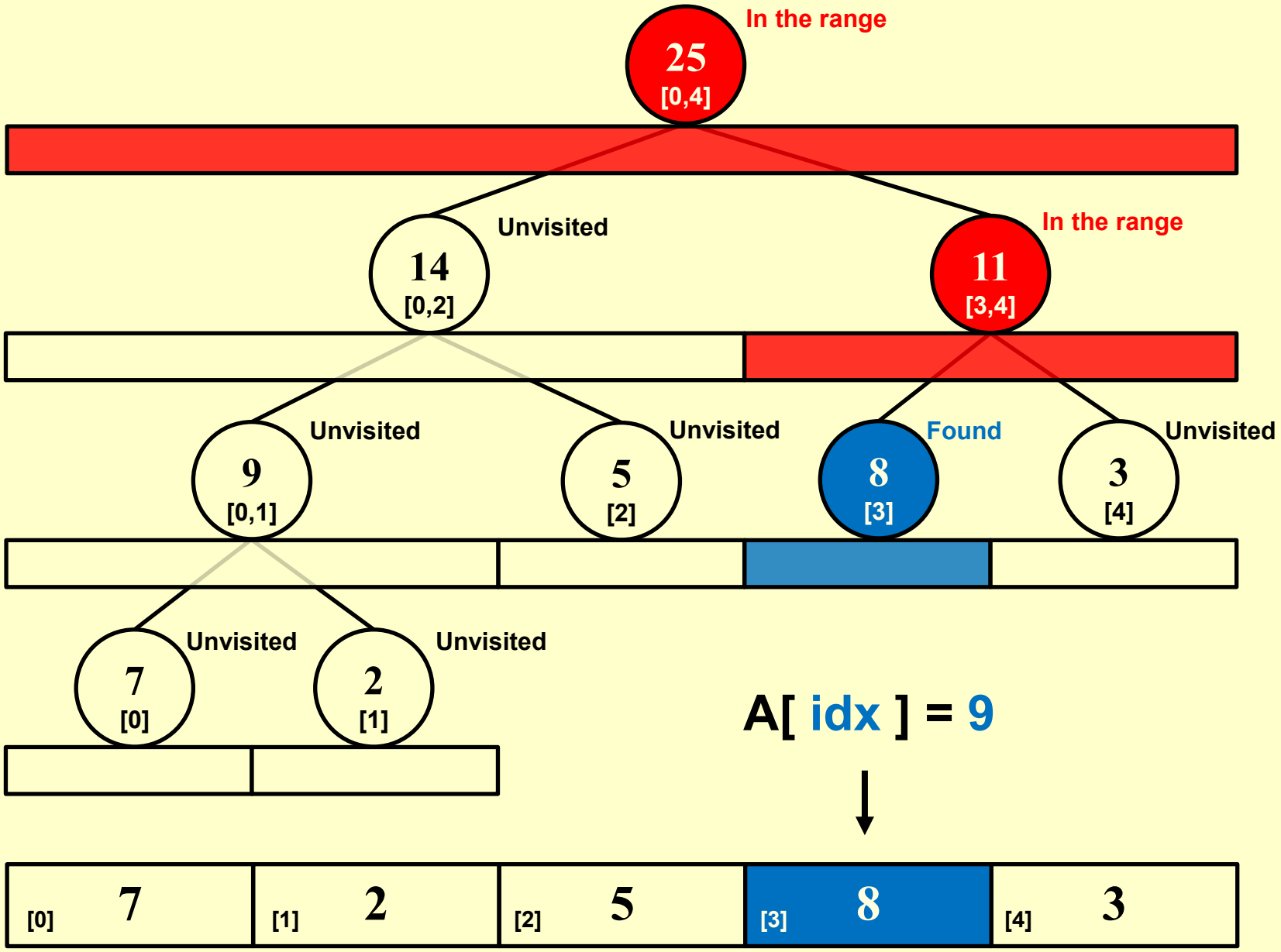


The Segment Tree – *query*

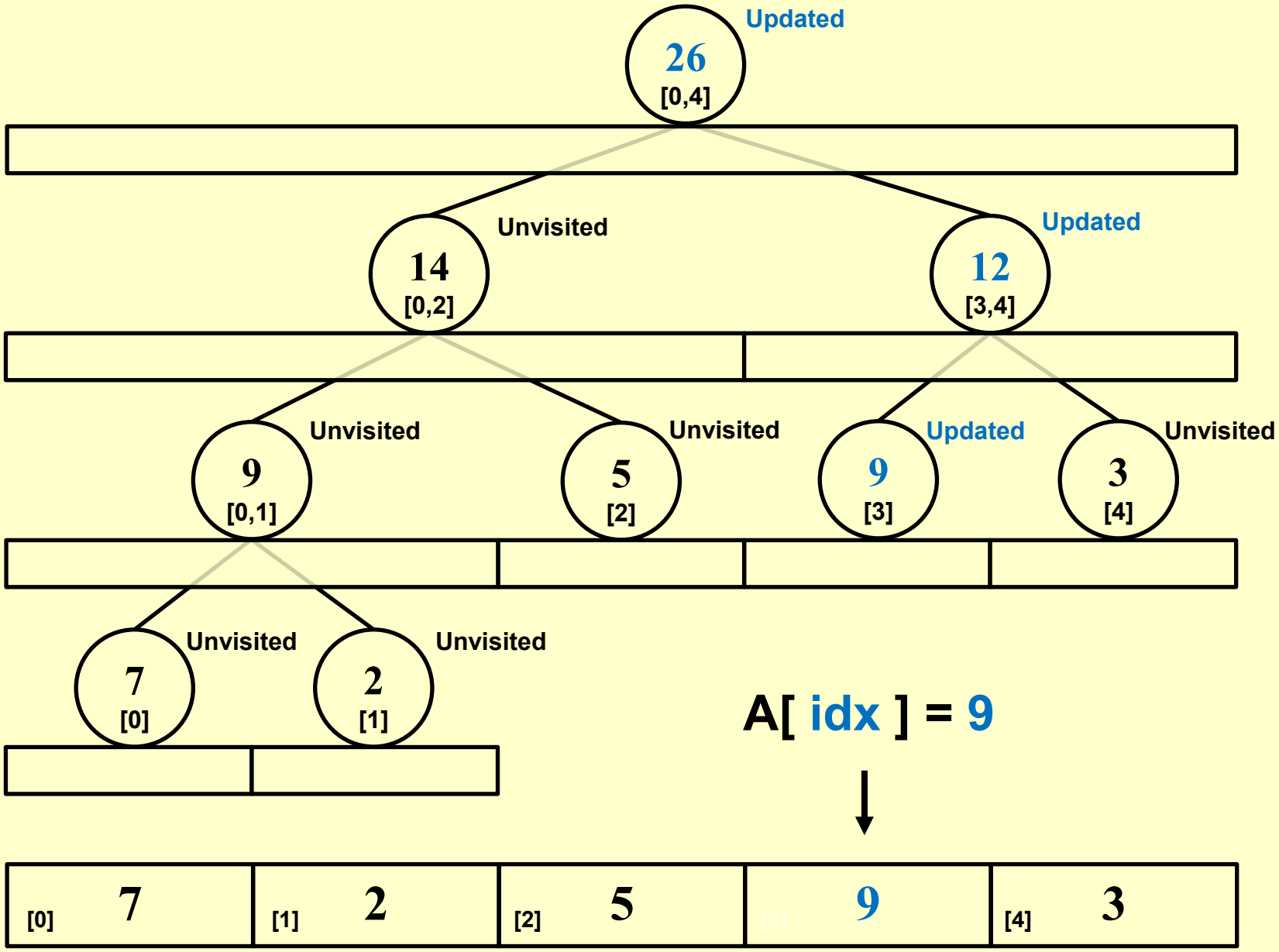
👍 $T(N) = O(\log N)$

```
int Query ( int node, int start, int end, int L, int R )
{
    // Case 1: No Overlap (Node is completely outside query range)
    if ( R < start || end < L ) {
        return 0;
    }

    // Case 2: Total Overlap (Node is completely inside query range)
    if ( L <= start && end <= R ) {
        return tree[ node ];
    }

    // Case 3: Partial Overlap (We need to go deeper into both children)
    int mid = ( start + end ) / 2;
    int left_sum = Query ( 2 * node, start, mid, L, R );
    int right_sum = Query ( 2 * node + 1, mid + 1, end, L, R );

    return left_sum + right_sum;
}
```

The Segment Tree – *update*



The Segment Tree – *update*



The Segment Tree – *update*

👍 $T(N) = O(\log N)$

```
void Update ( int node, int start, int end, int idx, int val )
{
    // Base Case: Leaf Node found
    if ( start == end ) {
        tree[ node ] = val;
        return;
    }

    // Recursive Step: Go left or right depending on where “idx” is
    int mid = (start + end) / 2;
    if ( start <= idx && idx <= mid ) {
        // Index is in the left child
        Update ( 2 * node, start, mid, idx, val );
    } else {
        // Index is in the right child
        Update ( 2 * node + 1, mid + 1, end, idx, val );
    }

    // Backtracking Step: Update current node based on new children values
    tree[ node ] = tree[ 2 * node ] + tree[ 2 * node + 1 ];
}
```

The Segment Tree – *notes*

1. **Not limited to sum, but applicable to any *aggregation operator* over a *range*, such as min, max, or average.**
2. **For more general RangeUpdate (e.g., add 10 to all numbers from index 2 to 1000), the *lazy propagation* strategy is effective (self study).**